Experiment No:03

Title: Perform Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, and Merge Sort using C

Aim:

To implement and understand different sorting algorithms in C: Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, and Merge Sort.

Mapped to Course Objective: CO1 Applications:

- Searching in databases Dictionary lookup File indexing systems

Theory:

Sorting algorithms are fundamental for organizing data. Each sorting technique follows a specific approach:

- Bubble Sort: Repeatedly swaps adjacent elements if they are in the wrong order.
- Selection Sort: Selects the minimum element and places it at the correct position.
- Insertion Sort: Builds sorted array by inserting elements at correct position.
- Quick Sort: Divide and conquer algorithm using a pivot element.
- Merge Sort: Divides array into halves, recursively sorts and merges them.

Algorithms:

- Bubble Sort: Compare adjacent elements and swap if needed, repeat until array is sorted.
- Selection Sort: Select the smallest (or largest) element and place it in correct position.
- Insertion Sort: Pick elements and insert them at the right position in sorted part of array.
- Quick Sort: Partition the array using a pivot, and apply quicksort to subarrays.
- Merge Sort: Recursively split the array and merge sorted subarrays

```
#include<stdio.h>

void display(int arr[],int n){
   int i;
   for(i=0;i<n;i++){
      printf("%d ",arr[i]);
   }
   printf("\n");
}
```

```
void bubbleSort(int arr[],int n){
   int i,j,temp;
   for(i=0;i<n-1;i++){
      for(j=0;j<n-i-1;j++){
         if(arr[j]>arr[j+1]){
            temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
         }
      }
   }
}

void selectionSort(int arr[],int n){
   int i,j,min,temp;
   for(i=0;i<n-1;i++){
      min=i;
      for(j=i+1;j<n;j++){
         if(arr[j]<arr[min]){
            min=j;
         }
      }
      temp=arr[i];
      arr[i]=arr[min];
      arr[min]=temp;
   }
}

void insertionSort(int arr[],int n){
   int i,j,key;
   for(i=1;i<n;i++){
      key=arr[i];
      j=i-1;
      while(j>=0 && arr[j]>key){
         arr[j+1]=arr[j];
         j--;
      }
      arr[j+1]=key;
   }
}
```

```
void quickSort(int arr[],int low,int high){
   int i,j,pivot,temp;
   if(low<high){
      pivot=arr[high];
      i=low-1;

      for(j=low;j<high;j++){
         if(arr[j]<pivot){
            i++;
            temp=arr[i];
            arr[i]=arr[j];
            arr[j]=temp;
         }
      }

      temp=arr[i+1];
      arr[i+1]=arr[high];
      arr[high]=temp;

      quickSort(arr,low,i);
      quickSort(arr,i+2,high);
   }
}

void merge(int arr[],int low,int mid,int high){
   int temp[100],i,j,k;
   i=low;
   j=mid+1;
   k=low;

   while(i<=mid && j<=high){
      if(arr[i]<arr[j]){
         temp[k]=arr[i];
         i++;
      }
      else{
         temp[k]=arr[j];
         j++;
      }
      k++;
```

```
    }

    while(i<=mid){
        temp[k]=arr[i];
        i++;
        k++;
    }

    while(j<=high){
        temp[k]=arr[j];
        j++;
        k++;
    }

    for(i=low;i<=high;i++){
        arr[i]=temp[i];
    }
}

void mergeSort(int arr[],int low,int high){
    int mid;
    if(low<high){
        mid=(low+high)/2;
        mergeSort(arr,low,mid);
        mergeSort(arr,mid+1,high);
        merge(arr,low,mid,high);
    }
}

int main(){
    int arr[100],n,i,choice;

    printf("Enter number of elements: ");
    scanf("%d",&n);

    printf("Enter array elements: ");
    for(i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }

    printf("\n1. Bubble Sort");
```

```
    printf("\n2. Selection Sort");
    printf("\n3. Insertion Sort");
    printf("\n4. Quick Sort");
    printf("\n5. Merge Sort");

    printf("\nEnter your choice: ");
    scanf("%d",&choice);

    switch(choice){
        case 1:
            bubbleSort(arr,n);
            break;
        case 2:
            selectionSort(arr,n);
            break;
        case 3:
            insertionSort(arr,n);
            break;
        case 4:
            quickSort(arr,0,n-1);
            break;
        case 5:
            mergeSort(arr,0,n-1);
            break;
        default:
            printf("Invalid choice");
            return 0;
    }

    printf("\nSorted array: ");
    display(arr,n);

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter array elements: 5 2 8 1 4

1. Bubble Sort
2. Selection Sort
```

3. Insertion Sort
4. Quick Sort
5. Merge Sort
Enter your choice: 1

Sorted array: 1 2 4 5 8